EXPERIMENT 06

Deluge workflows testing and debugging

Aim

Implement circulation validation, return calculations and reminder workflows, then test both successful and failed paths.

Procedure

1. Create a Loans workflow for record creation, On Validate. Required Copy and Member fields must be configured first. Paste and compile the illustrative script below using the exact link names from Experiment 4.

2. Keep this workflow free of email and outbound API calls. Failed validation should not send messages or create remote records. Put external effects after a successful save or in a retryable scheduled process.

3. Restrict all ordinary users from editing the core loan identity and status directly. Confirm that API and import paths also enforce the same rules and cannot bypass workflows.

```
// Loans: Created -> On Validate
memberRow = Members[ID == input.Member];
copyRow = Book_Copies[ID == input.Copy];
if(memberRow.count() == 0 || copyRow.count() == 0)
{
    alert "Select an existing member and copy.";
    cancel submit;
}
if(memberRow.Active != true)
{
    alert "This member is inactive.";
    cancel submit;
}
if(copyRow.Service_Status != "In Service")
{
    alert "This copy is not in service.";
    cancel submit;
}
openForCopy = Loans[Copy == input.Copy && Status == "Issued"].count();
openForMember = Loans[Member == input.Member && Status == "Issued"].count();
if(openForCopy > 0 || openForMember >= 3)
{
    alert "Copy unavailable or member loan limit reached.";
    cancel submit;
}
input.Issue_Date      =      zoho.currentdate;
input.Due_Date = zoho.currentdate.addDay(14);
input.Status = "Issued";
input.Return_Date = null;
input.Fine_Amount = 0;
```

Boundary: this script illustrates sequential validation. Two concurrent requests can both pass the counts. Production acceptance requires atomic allocation or enforced serialization, including API and import traffic.

Procedure continued

1. Create a controlled return path for librarians. Verify the saved loan is Issued, require a return date on or after Issue_Date and no later than today, and reject repeat returns. Enforce role authorization and unchanged Copy, Member, Issue_Date and Due_Date before running the calculation.

Implementation and verification

2.Use the following calculationin the controlled Loans editOn Validate workflow, after those guards. The return date is the actual return date, not the date on which an overdue report is viewed.

```
// Controlled return edit: validation and authorization run first.
if(input.Return_Date == null)
{

    alert "Enter the return date.";
    cancel submit;
}
if(input.Return_Date < input.Issue_Date ||
   input.Return_Date > zoho.currentdate)
{
    alert "Return date is outside the allowed range.";
    cancel submit;
}
lateDays = 0;
if(input.Return_Date  >  input.Due_Date)
{
    lateDays = input.Due_Date.daysBetween(input.Return_Date);
}
input.Fine_Amount = lateDays * 2;
input.Status = "Returned";
```

Do not expose this fragment as an unrestricted edit handler. A complete handler must compare the submitted record with the saved loan, reject invalid state transitions and prevent changes to historical fines. Administrative corrections need a separate audited process.

1. Configure a daily scheduled workflow in the app timezone, for example 09:00. Select issued loans due before today; process a bounded batch and skip any with Last_Reminder_On equal to today.

2. Resolve the member email from the lookup. In testing, route messages only to an authorized test mailbox. Send a short reminder containing loan number, title and due date, then set Last_Reminder_On only after a successful send.

3. For larger workloads, create a notification queue with a unique key based on Loan ID and reminder date, a status and retry count. A simple timestamp check reduces repeats but does not guarantee exactly-once delivery after a crash.

4. Inspect execution logs for failures; add temporary info statements containing record IDs and counts only. Reproduce the input, correct the root cause, retest, and remove sensitive debug output.

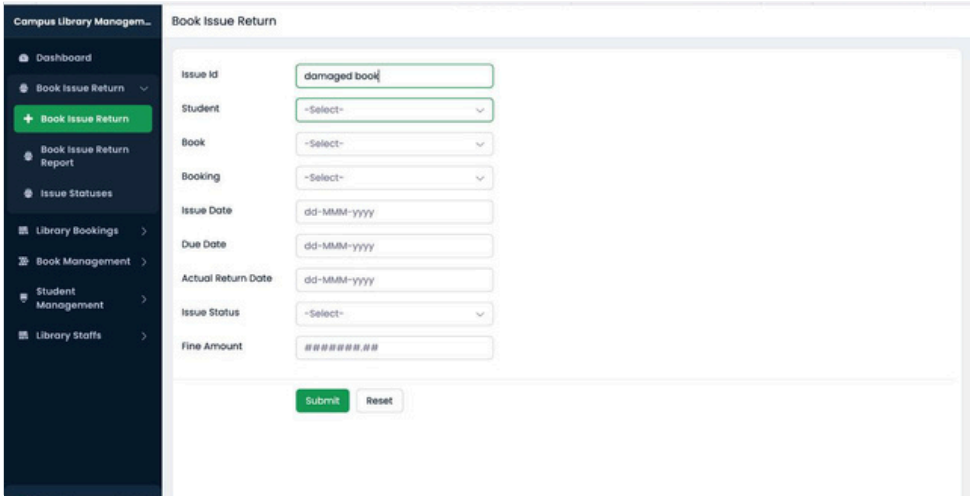
Procedure continued

Essential workflow checks

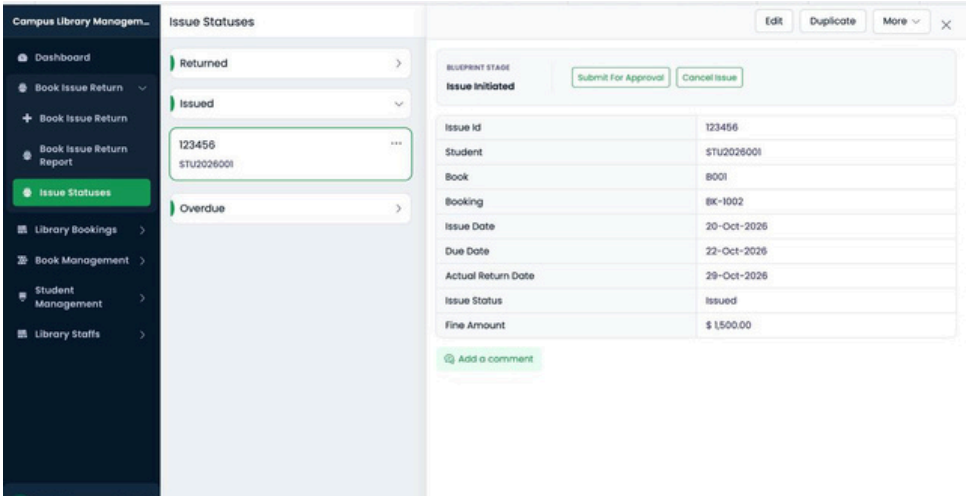
T01 valid issue; T02 duplicate-copy rejection; T03 three-loan limit; T04 inactive member or maintenance copy; T05 return on due date with INR 0 fine; T06 return three days late with INR 6 fine; T07 invalid return date; T08 repeat return; T09 reminder rerun; T10 simultaneous issues; T11 API or import validation; T12 unauthorized edit. Capture actual outcomes. Any concurrent double issue blocks production use.

Application images and explanation

Book Issue entry form



Book Return entry form



Supplied application screenshot from the uploaded library document.

The Book Issue and Book Return screenshots show the available form interfaces. They do not show the Deluge code, its event binding, logs or negative-test results. Use the written tests to verify the behavior behind those screens before claiming that the workflow passed.

Result

Expected result and verification criteria. Required sequential validation, return rules and reminder behavior pass their test cases with captured evidence. Any race condition or workflow bypass blocks production approval until it is corrected and retested.